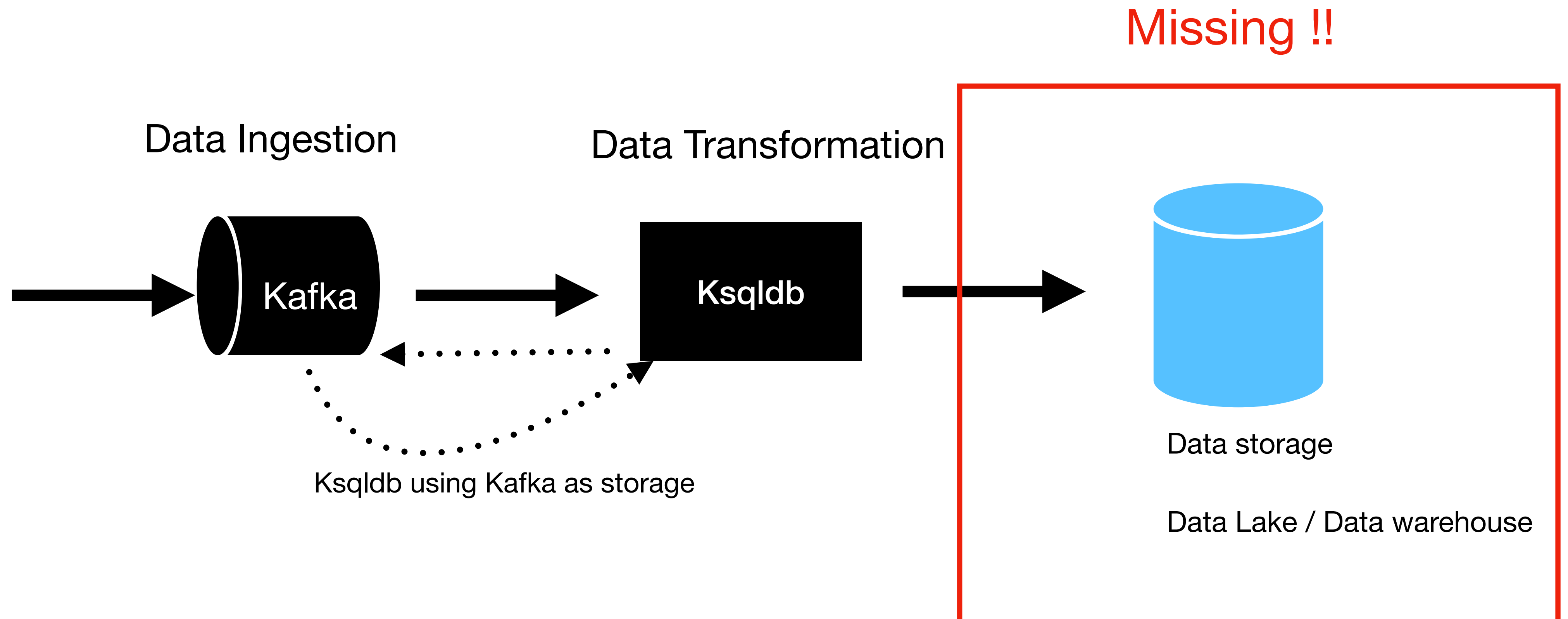


De ksqlDB au Data Lake/ warehouse

Lionel Souop

Data Integration with streaming

Where we are

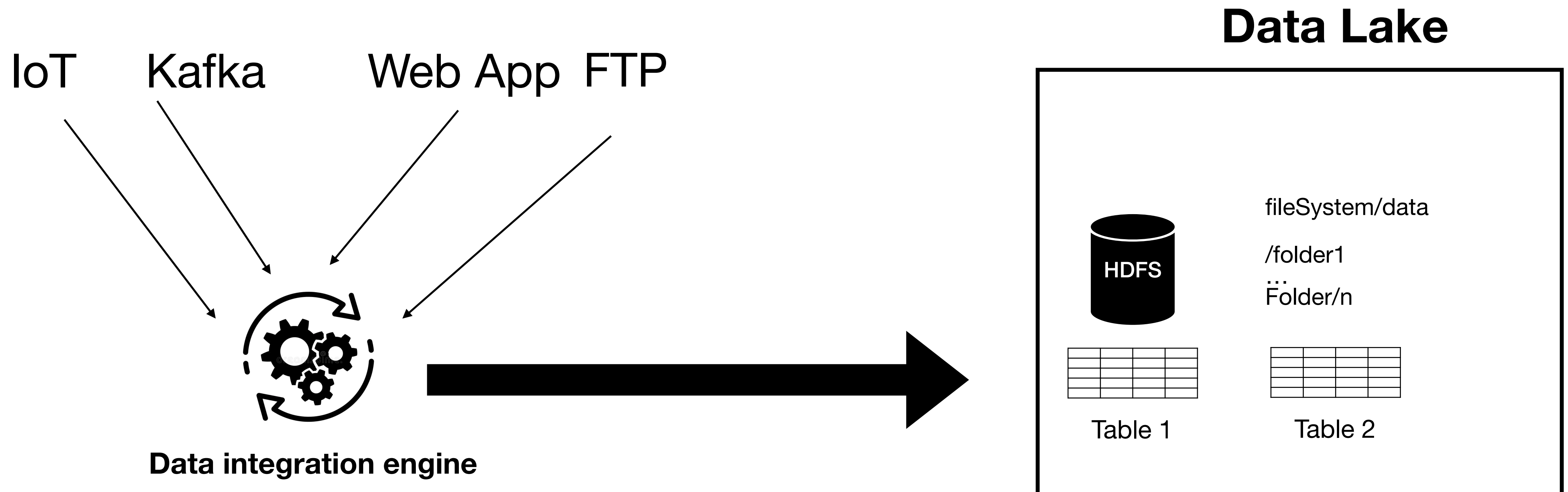


Data Lake

Data Lake

Définition

- Un data lake est un espace de stockage, où de grandes quantités de données, provenant de différentes sources, peuvent être stockées, généralement dans leur format brut ou très peu transformées.



Data Lake

Définition

- Un data lake n'a pas de structure prédéfinie, ne s'appuie pas sur des technologies prédéfinies, mais se construit selon le besoin.
- Un data lake peut donc être une combinaison d'une ou plusieurs des solutions suivantes:
 - HDFS
 - Base de données
 - Dossiers sur un espace de stockage
 - Etc ...
- Il existe cependant des patrons d'architectures de data lake pour guider dans la mise en place d'une telle infrastructure: Data Mesh, Lambda Architecture, médaillon storage etc ...

Data Lake

Définition

- Un data lake n'a pas de structure prédéfinie, ne s'appuie pas sur des technologies prédéfinies, mais se construit selon le besoin.
- Un data lake peut donc être une combinaison d'une ou plusieurs des solutions suivantes:
 - HDFS
 - Base de données
 - Dossiers sur un espace de stockage
 - Etc ...
- Il existe cependant des patrons d'architectures de data lake pour guider dans la mise en place d'une telle infrastructure: Data Mesh, Lambda Architecture, médaillon storage etc ...
- L'objectif est de permettre un stockage centralisé, où toutes les données d'une entreprise ou d'un système peuvent être stockées, qu'elles soient structurées ou non, pour être ensuite traitées et analysées à la demande.

Data Lake

Caractéristiques

1. **Stockage brut** : Le data lake peut accueillir n'importe quel type de donnée, structurée (bases de données, CSV), semi-structurée (JSON, XML) ou non structurée (texte, images, vidéos, logs, etc.).
2. **Évolutivité** : Il peut gérer des volumes massifs de données et peut évoluer avec l'augmentation de ces volumes (ex : stockage dans des systèmes comme HDFS, S3, ou Azure Data Lake).
3. **Accessibilité à long terme** : Les données restent stockées pendant longtemps, ce qui permet aux entreprises d'y revenir pour effectuer des analyses sans devoir repasser par le processus de collecte initial.
4. **Schéma flexible** : Contrairement à un data warehouse (où les données doivent suivre un schéma strict avant d'être insérées), dans un data lake, les données peuvent être stockées telles qu'elles sont.

Data Lake

Limites

Complexité de gestion : Si les données ne sont pas bien organisées dès le début, un data lake peut devenir un "lac de boue" où il devient difficile de retrouver ou comprendre les données.

Manque de gouvernance : Comme les données sont stockées de manière brute, il faut une bonne stratégie de gestion des métadonnées, de sécurité et de contrôle d'accès.

Data Lake

Exemples d'outils de Data Lake

- **HDFS (Hadoop Distributed File System)** : Une solution de stockage distribuée open-source.
- **Amazon S3, Azure Data Lake, Google Cloud Storage** : Des solutions cloud pour stocker des données en grande quantité.
- **Apache Spark, Flink, KsqlDB** : Des outils pour traiter de grandes quantités de données en temps réel ou en batch.
- **Presto, Hive, Elasticsearch** : Des moteurs de requêtes distribués pour interroger les données dans le data lake.

Data warehouse

Data warehouse

Définition

Un Data Warehouse est une base de données centralisée, optimisée pour l'analyse des données. Il permet de stocker de grandes quantités de données provenant de différentes sources opérationnelles et de les structurer de manière à ce qu'elles soient facilement accessibles pour l'analyse décisionnelle.

Caractéristiques principales d'un Data Warehouse :

- **Intégration des données :** Il regroupe des données provenant de plusieurs sources (bases de données transactionnelles, systèmes externes, applications, etc.).
- **Historique des données :** Un Data Warehouse conserve des données historiques pour effectuer des analyses sur des périodes longues.
- **Optimisé pour la lecture :** Contrairement aux bases de données transactionnelles, le Data Warehouse est conçu pour des requêtes de lecture complexes plutôt que pour des écritures fréquentes.
- **Structuration des données :** Les données sont souvent nettoyées, transformées et organisées pour faciliter l'analyse.

Data warehouse

Avantages

- **Performance optimisée pour les requêtes analytiques** : Les Data Warehouses sont optimisés pour les requêtes complexes et les analyses sur de grandes quantités de données.
- **Décisions basées sur des données historiques** : Les entreprises peuvent analyser les tendances et les patterns sur de longues périodes pour prendre des décisions éclairées.
- **Centralisation des données** : Un Data Warehouse regroupe des données provenant de diverses sources, offrant ainsi une vue unifiée de l'entreprise.
- **Sécurité et gouvernance des données** : Les données peuvent être sécurisées et gérées de manière centralisée, ce qui facilite leur traçabilité et leur conformité.

Data warehouse

Limites

- **Coût** : Les Data Warehouses peuvent être coûteux à mettre en place, surtout en mode on-premise.
- **Problèmes de performance avec de grandes quantités de données** : À mesure que le volume de données augmente, les performances des requêtes peuvent être affectées, bien que les solutions modernes de Data Warehouses cloud aident à résoudre ce problème avec des ressources scalables.
- **Rétrocompatibilité** : Les Data Warehouses sont parfois difficiles à adapter aux nouvelles sources de données, comme les données non structurées (logs, vidéos, etc.).

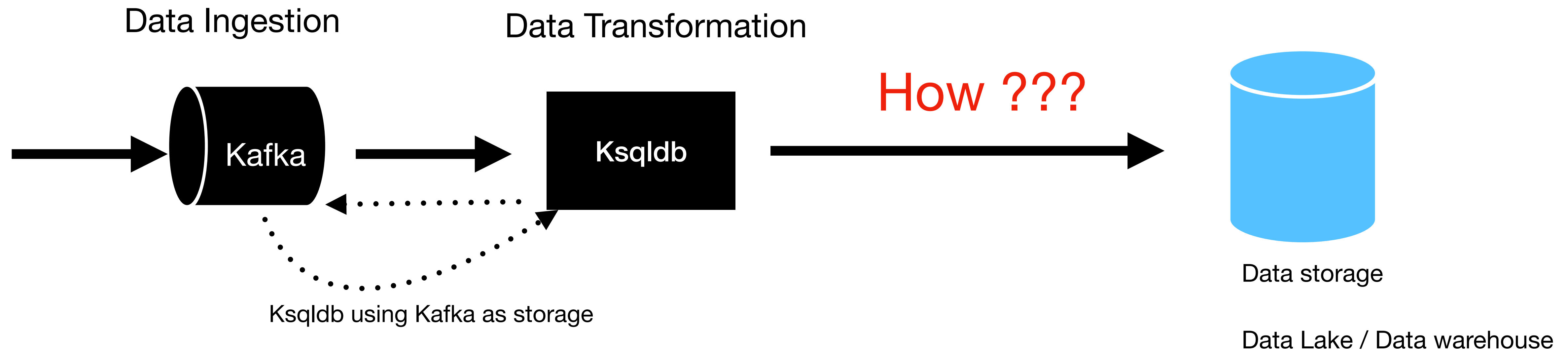
Data Lake vs warehouse

Critère	Data Lake	Data warehouse
Type de données	Non structurées, semi et structurées	Structurées uniquement
Cout	Moins cher, car stocke les données pas ou peu transformées	Plus cher, car nécessite un nettoyage préalable
Schéma	Schéma appliqué à la lecture ou écriture	Schéma appliqué lors de l'écriture
Outils d'analyse	Spark, presto, Flint, Hive	SQL, BI tools (power BI, tableau)
Cas d'utilisation	Big data, Machine learning, IoT, log, videos	Reporting, analyses historiques, BI

Ksqldb to Data Lake/warehouse

Ksqldb to data Lake

Goal



Ksqldb to data Lake

2 possible solutions

- Using a Kafka-connect connector
- Using an adhoc Kafka consumer application

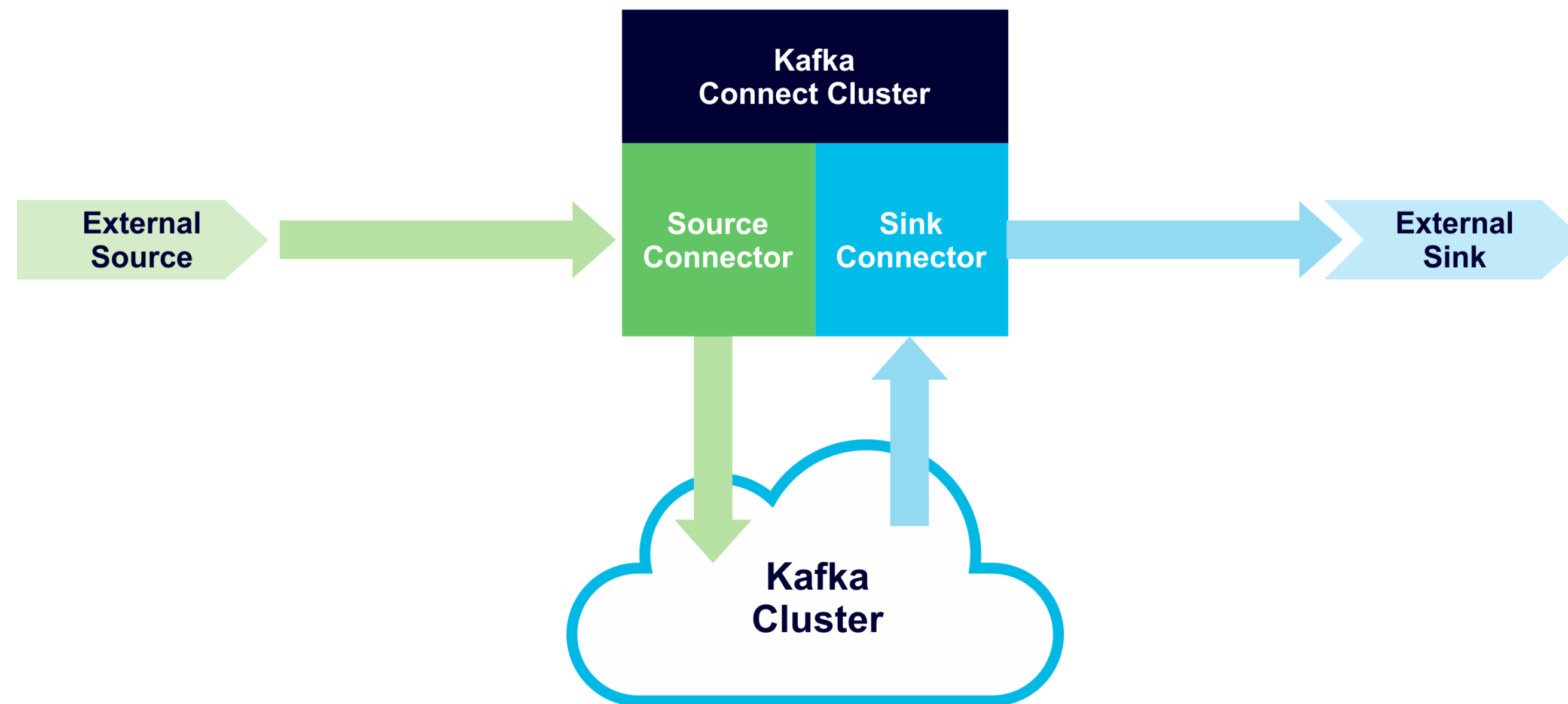
Ksqldb to data Lake

Using a Kafka-connect connector

- Kafka integrated solution that allow to connect Kafka to external sources and sink.
- Kafka connect is a separated cluster running on its own, using Kafka to ingest and sink data from/to external sources
- There're 2 types of connectors
 - Source connector: from external source to Kafka
 - Sink connector: from Kafka to external source

Ksqldb to data Lake

Architecture



Ksqldb to data Lake

Exemple of connectors

Connecteur	Type	Description
JDBC source	Source	Lit des données à partir de bases de données SQL
JDBC Sink	Sink	Écrit des données Kafka dans des bases SQL
Elasticsearch sink	Sink	Indexe les messages Kafka dans Elasticsearch
HDFS sink	Sink	Écrit les données dans Hadoop Distributed File System
S3 sink	Sink	Stocke les messages Kafka dans Amazon S3
Debezium	Source	Capture les changements de bases SQL (MySQL, PG...)
FileStream Source/ Sink	Les deux	Pour lire ou écrire dans des fichiers texte

Ksqldb to data Lake

Kafka connect pros and cons

- Pros

- Pas de code personnalisé
- Scalable
- Déploiement de configuration facile
- Tous les avantages des systèmes distribués

- Cons

- Complexe la mise en place the connect cluster
- Performances parfois limitée
- Limité en terme d'ajout de logique métier
- Support de différent format limité

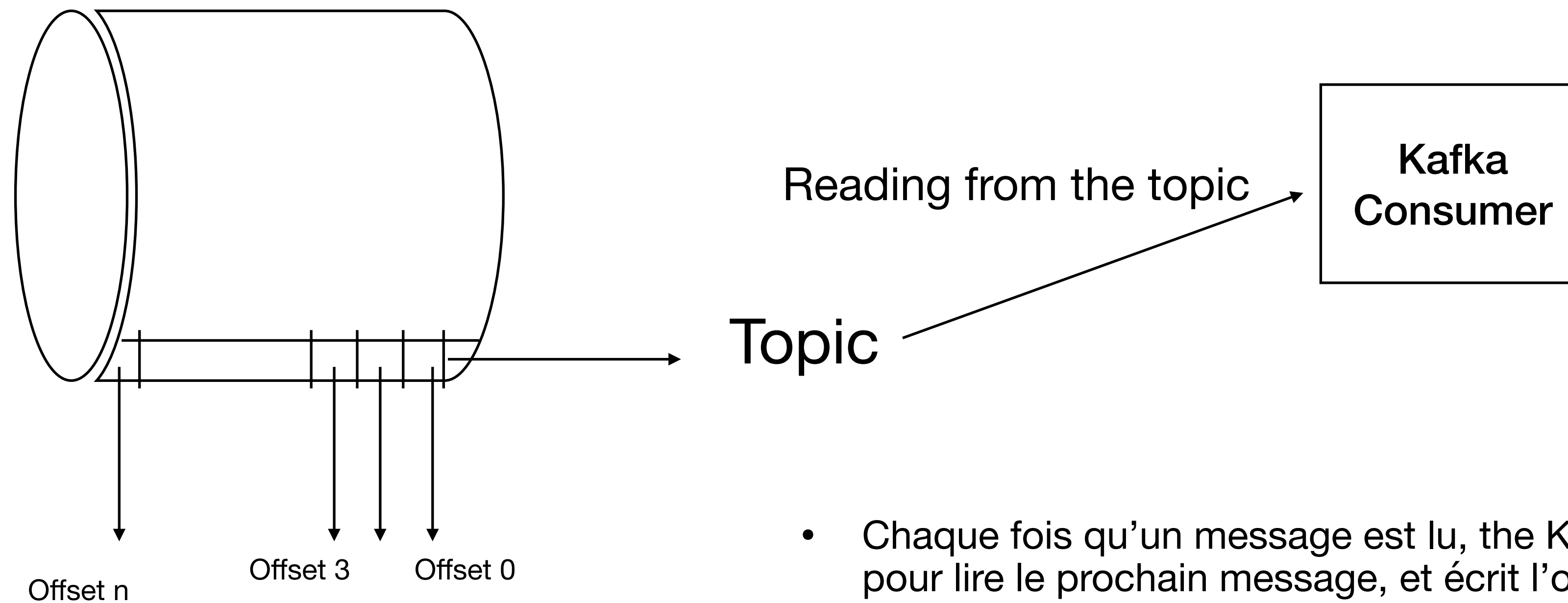
Ksqldb to data Lake

With a Kafka consumer

- Un Kafka Consumer est un client Kafka qui lit les messages d'un ou plusieurs topics Kafka. Il est utilisé pour consommer les données envoyées par les producteurs, souvent dans le but de les traiter, les stocker, ou les transmettre ailleurs.
- Composants clés d'un consumer Kafka
 - **Topic** : Conteneur logique des messages (regroupement).
 - **Partition** : Division d'un topic (le consumer lit une ou plusieurs partitions).
 - **Consumer Group** : Groupe de consumers qui se répartissent les partitions entre eux (scalabilité).
 - **Offset** : Position dans la partition (le consumer sait où il en est dans la lecture).

Ksqldb to data Lake

With a Kafka consumer



- Chaque fois qu'un message est lu, the Kafka consumer incrémenté l'offset pour lire le prochain message, et écrit l'offset dans des meta données.
- En cas de crash du Consumer, au démarrage, il lit à partir du dernier offset lu

Ksqldb to data Lake

Kafka consumer - mode d'écriture sur File System

Append

Définition : Ajoute les nouvelles données à la suite des données existantes.

Cas d'usage : Historiser des événements, logs, ventes, etc.

 **Avantage** : Aucune perte de données.

 **Risque** : Doublons si les mêmes données sont écrites plusieurs fois.

Ksqldb to data Lake

Kafka consumer - mode d'écriture sur File System

Overwrite

Définition : Supprime les données existantes et réécrit avec les nouvelles.

Cas d'usage : Mettre à jour des données de référence (par exemple, la table des produits à jour chaque nuit).

 **Avantage** : Aucune donnée obsolète.

 **Risque** : Perte des données précédentes si non sauvegardées ailleurs - Temps de consolidation qui peut être long

Ksqldb to data Lake

Kafka consumer - mode d'écriture sur File System

Ignore

Définition : Si les données existent déjà (ex : fichier, table, clé), ne fait rien du tout.

Cas d'usage : Éviter de surcharger une cible déjà écrite.

✅ **Avantage** : Protège les données existantes.

❌ **Risque** : échec d'écriture pas bien géré peut entrainer la perte de données - surcharge de la destination (bottleneck)

Ksqldb to data Lake

Kafka consumer - mode d'écriture sur File System

ErrorIfExists (ou simplement Error)

Définition : Provoque une erreur si les données existent déjà dans la destination.

Cas d'usage : T'assurer de ne pas écraser ou modifier des données critiques.

✅ **Avantage** : Sécurité, pas d'écrasement par accident.

❌ **Risque** : Le job plante si la cible est déjà présente.